Lecture 19: Structured Streaming I (Concepts)

DSL351 / DSP352: Big Data Analytics

Amit Kumar Dhar

IIT Bhilai

April 6, 2026

Introduction to Stream Processing

What is Stream Processing?

- **Batch Processing:** Processing a large, static dataset all at once.
- **Stream Processing:** Processing data that is continuously generated, often in real-time.
- Data is often referred to as **Unbounded**: there is no defined start or end to the stream.
- **Goal:** Low-latency insights from data as it arrives.

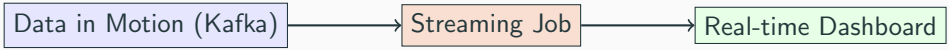
The Need for Speed

- In many industries, the value of data diminishes over time.
- **Operational Intelligence:** Reacting to events within seconds (e.g., stopping a server before it overheats).
- **Customer Experience:** Real-time recommendations while a user is browsing.
- **Financial Compliance:** Detecting money laundering patterns as transactions happen.

Batch vs. Stream Processing

Batch: High Latency, High Throughput

Streaming: Low Latency, Continuous



Latency vs. Throughput Trade-off

- **Latency:** Time taken to process a single record.
- **Throughput:** Number of records processed per second.
- Batch systems maximize throughput by processing records in bulk.
- Traditional stream systems minimize latency by processing records one-by-one.
- Spark Structured Streaming aims for a sweet spot using the **Micro-batch model**.

Real-world Use Cases

- **Fraud Detection:** Analyzing credit card transactions in milliseconds.
- **IoT Monitoring:** Monitoring temperature sensors in a factory.
- **Ad Tech:** Real-time bidding and click-stream analysis.
- **Log Analytics:** Monitoring system health across servers.
- **Social Media:** Live sentiment analysis.

Challenges in Stream Processing

- **Late Data:** Data may arrive out of order due to network delays.
- **Fault Tolerance:** Recovering if a node fails without losing data.
- **Exactly-once Semantics:** Ensuring every event is processed exactly once.
- **State Management:** Maintaining counters or windows across hours of data.
- **Load Balacing:** Handling spikes in traffic.

Structured Streaming Philosophy

The Evolution of Spark Streaming

1. **Spark Streaming (DStreams):**
   - Introduced in Spark 0.7.
   - Built on top of RDDs (Micro-batches of RDDs).
   - No native support for event-time.
   - Hard to interoperate with DataFrames/SQL.
2. **Structured Streaming (Spark 2.0+):**
   - Built on Spark SQL engine.
   - Uses DataFrames and Datasets.
   - Unified API for batch and streaming.

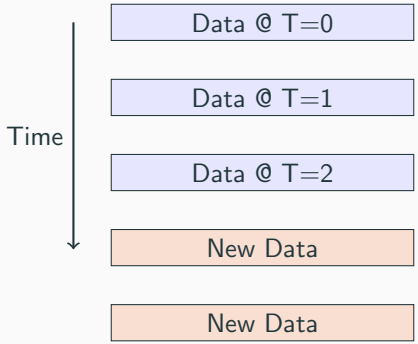
Why "Structured"?

- **Typed Data:** Schema-based processing allows for better optimizations.
- **SQL Engine:** Leverages Catalyst for query optimization and Tungsten for memory management.
- **Easier Logic:** Focus on "What" to compute rather than "How" to handle streams.

Core Idea: The Unbounded Table

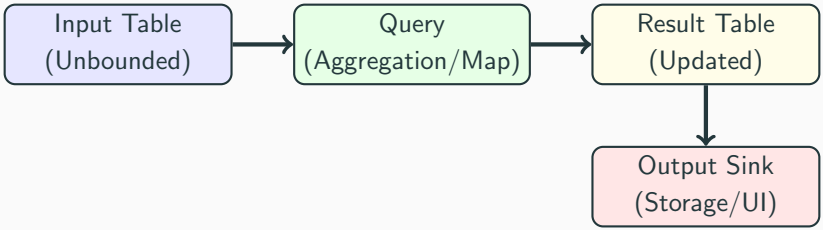
- Treats a live data stream as a table that is being **continuously appended**.
- Every new row in the stream is a row in the table.
- A query on the stream is like a query on a static table.
- Result Table is updated incrementally.

Unbounded Table: Conceptual Flow



Input Table grows indefinitely

The Programming Model: Query to Result



Incremental Execution

- Spark doesn't re-read the entire history for every trigger.
- It identifies the **delta** (new data) and updates the Result Table accordingly.
- This is possible because Catalyst understands the semantics of operations like `count()` or `sum()`.

Unified API: Same Code, Different Source

- **Batch Query:**
  - `spark.read.parquet(...).groupBy(...).count()`
- **Streaming Query:**
  - `spark.readStream.parquet(...).groupBy(...).count()`
- The logic remains identical!

Exactly-once Semantics

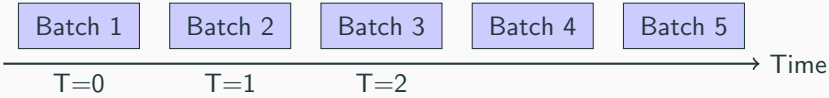
- Structured Streaming uses **Checkpoints** and **Write-Ahead Logs (WAL)**.
- It records the offset of data processed from the source.
- If a crash occurs, it restarts from the last recorded offset.
- Sinks must be **Idempotent** to ensure exactly-once delivery.

The Micro-batch Processing Model

How Spark Executes Streams

- Default execution engine: **Micro-batch Processing**.
- Data is accumulated into small batches (e.g., 500ms intervals).
- Each batch is processed as a standard Spark job.
- Provides high throughput and handles state robustly.

Micro-batch Visualized



Trigger Intervals

- **Default:** Run the next batch as soon as the previous one finishes.
- **Fixed Interval:** Run every N seconds/minutes.
- **One-Time (Once):** Run once and stop.
- **Continuous (Experimental):** Record-by-record processing (lowest latency).

Choosing a Trigger

- Use **ProcessingTime("10 seconds")** if you want predictable dashboard updates.
- Use **Trigger.Once** for cost-effective batch jobs that process streaming sources (e.g., daily ETL).
- Default is best for maximum speed.

Continuous Processing Mode

- Introduced in Spark 2.3.
- Sub-millisecond end-to-end latency.
- Uses long-running tasks on executors that continuously poll the source.
- Limited to simple map-like operations (no aggregations yet).

Micro-batch vs. Continuous Execution

Feature	Micro-batch	Continuous
Latency	100ms - 2s	1ms - 10ms
Throughput	Very High	High
Fault Tolerance	Checkpointing	Epoch Markers
Complex Aggs	Supported	Not Supported

Programming Model: Reading and Writing

Steps to Create a Streaming Query

1. **Define Source:** `spark.readStream`.
2. **Apply Transformations:** DataFrames operations.
3. **Define Sink:** `df.writeStream`.
4. **Configure Output:** Mode, Checkpoint, Trigger.
5. **Start Query:** `query.start()`.

Basic Structure: Scala

```scala
import org.apache.spark.sql.functions._

// 1. Read from Source
val lines = spark.readStream
  .format("socket")
  .option("host", "localhost")
  .option("port", 9999)
  .load()

// 2. Transform (Word Count)
val words = lines.as[String].flatMap(_.split(" "))
val wordCounts = words.groupBy("value").count()

// 3. Write to Sink
val query = wordCounts.writeStream
  .outputMode("complete")
  .format("console")
  .start()

query.awaitTermination()
```

Basic Structure: Python

```python
from pyspark.sql.functions import explode, split

# 1. Read from Source
lines = spark.readStream \
    .format("socket") \
    .option("host", "localhost") \
    .option("port", 9999) \
    .load()

# 2. Transform
words = lines.select(
   explode(split(lines.value, " ")).alias("word")
)
word_counts = words.groupBy("word").count()

# 3. Write to Sink
query = word_counts.writeStream \
    .outputMode("complete") \
    .format("console") \
    .start()

query.awaitTermination()
```

Streaming Sources

- **File Source:** Parquet, JSON, CSV. Reads new files in a folder.
- **Kafka Source:** Standard for production pipelines.
- **Socket Source:** For local testing only.
- **Rate Source:** Generates dummy data for testing performance.

Specifying Schemas

- Unlike batch, Spark cannot "infer" the schema of a stream because data is arriving.
- You **must** provide the schema for file-based sources.
- Kafka source has a fixed schema (key, value, topic, partition, offset).

Example: JSON Stream with Schema (Python)

```python
from pyspark.sql.types import StructType, StringType, IntegerType

userSchema = StructType() \
    .add("name", StringType()) \
    .add("age", IntegerType())

df = spark.readStream \
    .schema(userSchema) \
    .json("/path/to/json/directory")
```

Output Modes

1. **Append (Default):** Only new rows added to the Result Table are output.
2. **Complete:** The entire Result Table is re-written to the sink.
3. **Update:** Only rows that were modified in the Result Table are output.

Output Mode: Detailed Scenarios

- **Append:** Used for simple filters and maps. If you have an aggregation, Append is only allowed with watermarking!
- **Complete:** Used for all aggregations. It keeps the entire state visible.
- **Update:** Efficient for aggregations where you only want to know the "changed" counters.

Output Mode Table

Mode	Query Type	Outcome
Append	Selection/Filter	Only new records written.
Complete	Aggregation	All groups written every time.
Update	Aggregation	Only changed groups written.

Streaming Sinks

- **File Sink:** Parquet, ORC, etc.
- **Kafka Sink:** Send data to another topic.
- **Console Sink:** Debugging.
- **Memory Sink:** Store in memory for SQL queries.
- **ForeachBatch:** Most flexible (write to any DB).

Example: Kafka Sink (Scala)

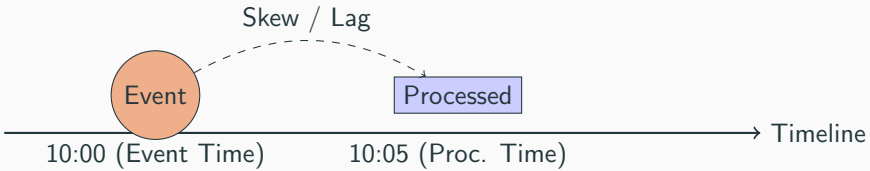
```scala
val query = resultDF
  .selectExpr("CAST(userId AS STRING) AS key", "to_json(struct(*)) AS value")
  .writeStream
  .format("kafka")
  .option("kafka.bootstrap.servers", "host:9092")
  .option("topic", "output-topic")
  .option("checkpointLocation", "/path/to/checkpoint")
  .start()
```

Windows on Event Time

Event Time vs. Processing Time

- **Processing Time:** When Spark gets the data.
- **Event Time:** When the data was created (timestamp in data).
- Critical for accurate analytics (e.g., user activity at midnight).

Processing vs. Event Time Diagram



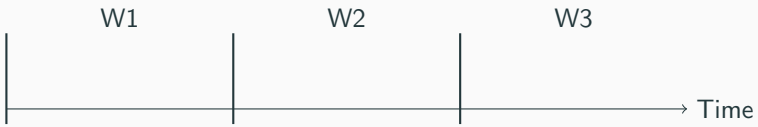
Windowing Concepts

- Aggregating data over time intervals.
- **Tumbling Windows:** Fixed-size, non-overlapping.
- **Sliding Windows:** Overlapping, fixed-size.

1. Tumbling Windows

- No gaps, no overlaps.
- Example: "Count requests every 5 minutes".
- Windows: [00:00, 00:05), [00:05, 00:10).

Tumbling Windows Visual



Tumbling Windows Code: Python

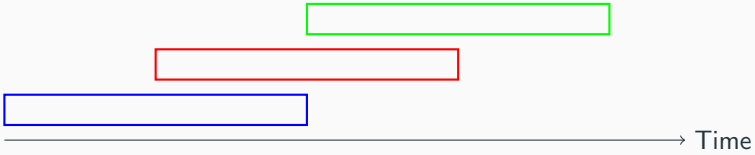
```python
from pyspark.sql.functions import window

windowedCounts = df \
    .groupBy(
        window(df.timestamp, "10 minutes"),
        df.category
    ) \
    .count()
```

2. Sliding Windows

- Windows overlap.
- Requires **Window Duration** and **Slide Duration**.
- Example: "10-minute window, starts every 5 minutes".

Sliding Windows Visual



Sliding Windows Code: Scala

```scala
val windowedCounts = df
  .groupBy(
    window(col("timestamp"), "10 minutes", "5 minutes")
  )
  .count()
```

Handling Late Data and Watermarks

The Problem of Late Data

- Network glitches cause events to arrive out of order.
- How long should Spark keep the state of a window open?
- If we wait forever, memory overflows.

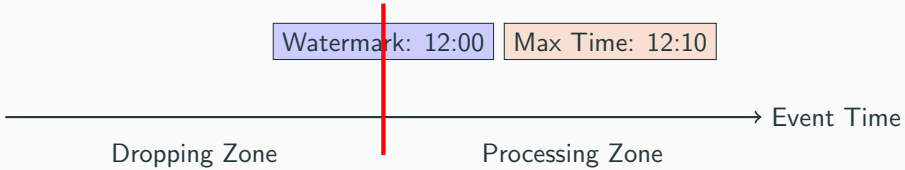
Watermarking: The Solution

- **Watermark:** A moving threshold defining the limit of "late data".
- Calculation: Watermark = max(Event Time seen) − Threshold.
- Data older than the watermark is dropped.

Watermarking Step-by-Step

1. System sees event @ 12:05. Threshold = 10m.
2. Watermark = 11:55.
3. Event arrives @ 11:58. $\rightarrow$ **Keep** (within 10m).
4. Event arrives @ 11:50. $\rightarrow$ **Drop** (older than 11:55).

Watermarking Visual



Watermarking Code: Python

```python
df.withWatermark("timestamp", "10 minutes") \
  .groupBy(window("timestamp", "10 minutes")) \
  .count()
```

State Cleanup with Watermarks

- When `Watermark > Window End Time`, Spark deletes the window state.
- This prevents memory OOM.
- Only works for **Append** and **Update** modes.

Cumulative Aggregations

- What if you want a global count?
- Use `Update` mode without windows.
- Spark maintains the state indefinitely.

Structured Streaming vs. DStreams (RDDs)

DStreams	Structured Streaming
RDD based	DataFrame based
Manual state handling	Automatic state management
Processing time only	Native Event Time/Watermarks
No SQL optimization	Catalyst/Tungsten optimized

Checkpoints and Fault Tolerance

- Essential for production.
- `.option("checkpointLocation", "/path")`.
- Stores query progress as JSON and HDFS files.
- Allows the job to resume exactly where it left off.

Checkpoint Internals

- **Offsets:** Which Kafka messages were read.
- **Commits:** Which batches were completed.
- **State:** Binary snapshots of window aggregates.

Production Checklist

1. Always use watermarking for stateful queries.
2. Always define a checkpoint location.
3. Monitor the Spark UI (Streaming tab).
4. Ensure sink is idempotent.
5. Match trigger interval to your latency needs.

Common Pitfalls

- **Infinitely growing state:** Forgetting watermarks.
- **Mixing sources:** Joins between two streams (Class 20).
- **Source data retention:** Kafka topic data deleted before Spark reads it.

Summary of Key Concepts

- Unbounded Table = Stream as a Table.
- Micro-batch = Core engine.
- Event Time = Truth in Big Data.
- Watermark = Memory management for late data.

Questions?